

Human Action Recognition for Human-Robot Interaction (HRI)

Alessandro Cortese & Luca Rossetto

September 2026

Abstract

Reliable Human Action Recognition (HAR) is a fundamental prerequisite for safe and effective Human-Robot Interaction (HRI) in collaborative environments. This report presents a framework designed to detect and classify six fundamental human actions (*boxing*, *hand clapping*, *hand waving*, *jogging*, *running*, and *walking*) across continuous 40-frame image sequences from the KTH Action Recognition dataset. We implement and evaluate two complete pipelines: a Classical Computer Vision pipeline using KNN background subtraction, vertical morphological filtering, dense Farnebäck optical flow vector directionality, and SVM classification; and a Deep Learning Fallback pipeline using a pretrained YOLOv8-Pose ONNX network for robust silhouette localization and pose-based feature extraction. Both pipelines feed into a multi-class Support Vector Machine (SVM) evaluated via 6-Fold Stratified CrossValidation. Experimental results demonstrate that while the classical approach achieves a Mean Intersection over Union (mIoU) of 0.7567 and a CV accuracy of 70.83%, the deep learning fallback overcomes classical limits under lighting and clothing variations, achieving an mIoU of 0.8544 and a CV accuracy of 93.06% (67 out of 72 sequences correctly classified).

1 Introduction and Project Goal

The main objective of this project is to develop a system capable of processing continuous 40-frame image sequences to perform two core tasks:

1. **Subject Localization and Tracking** : Isolate the human actor from static background clutter and compute a dynamic bounding box for each frame, evaluated via Intersection over Union (IoU) on the 20th frame (median frame).
2. **Kinematic Feature Extraction and Classification** : Analyze spatial-temporal motion dynamics across the 40 frame window, extract descriptive feature vectors, and classify the entire sequence with a single global semantic action label using a Support Vector Machine (SVM).

The framework is tested against 72 stratified sequences from the benchmark KTH Action Recognition Dataset across six action classes: *Boxing* (1), *Handclapping* (2), *Handwaving* (3), *Jogging* (4), *Running* (5), and *Walking* (6).

2 Dataset Stratification and Evaluation Protocol

To validate system generalization without environmental bias, a stratified dataset of 72 video clips was curated. Each action class contains 12 sequences symmetrically sampled across four distinct KTH recording scenarios:

- **Scenario d1**: Standard outdoor environment with uniform background and clear lighting.

- **Scenario d2:** Outdoor environment with scale variations, camera zoom shifts, and distance changes.
- **Scenario d3:** Outdoor environment with clothing variations.
- **Scenario d4:** Indoor environment with complex indoor illumination, low contrast, and cast shadows.

Each sequence folder contains exactly 40 pre extracted continuous image frames (160×120 pixels) and a ground-truth annotation text file containing the numerical class label and the reference ground-truth bounding box coordinates for the median frame (frame 20):

$$\langle \text{class_id} \rangle \langle \text{x_center} \rangle \langle \text{y_center} \rangle \langle \text{width} \rangle \langle \text{height} \rangle \quad (1)$$

3 System Architecture and Software Design

The code is written C++ using OpenCV and CMake. It follows an object-oriented design:

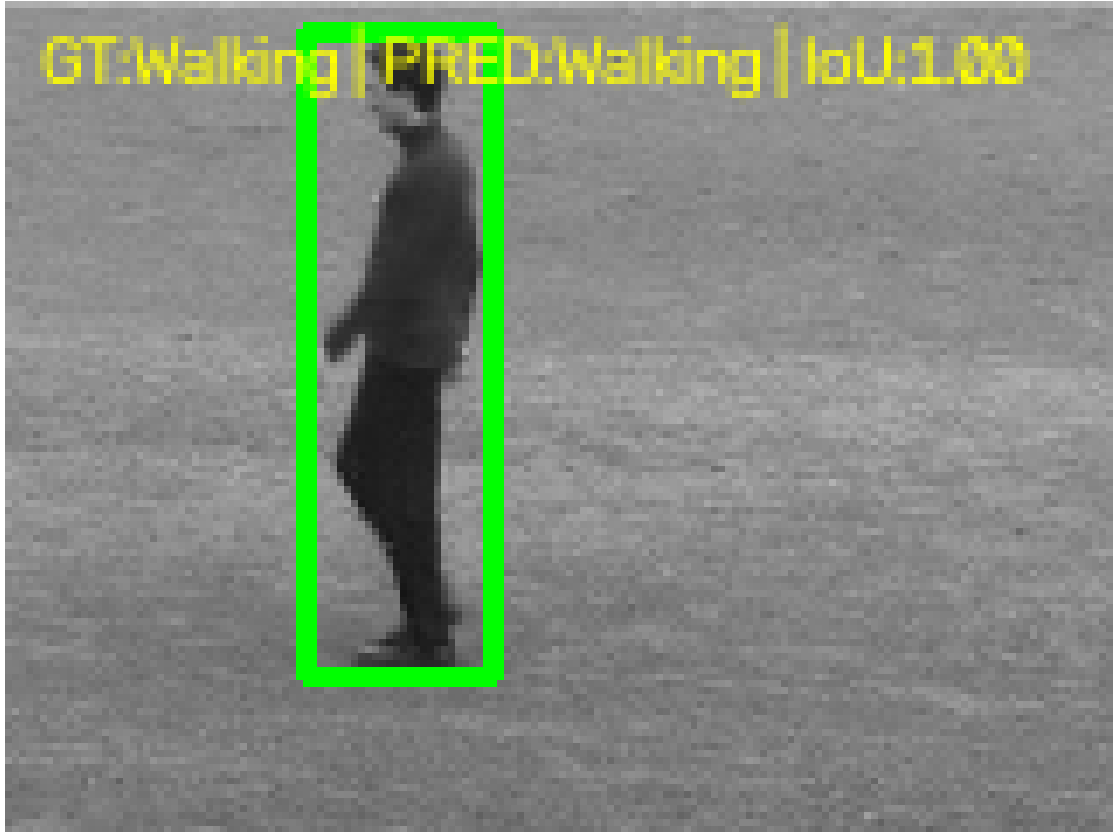


Figure 1: Example output visualization generated by the system, displaying the Ground Truth bounding box (blue), Predicted bounding box (green), predicted action label, and median frame IoU score.

- **DatasetLoader:** Reads sequence frames and parses normalized groundtruth bounding box text files.
- **Tracker (CV Mode):** Computes a sequence wide median background image $\mathbf{B}(x, y) = \text{median}_{t=1..N} \{\mathbf{I}_t(x, y)\}$ to eliminate moving subjects, applies absolute frame difference $|\mathbf{I}_t - \mathbf{B}|$, Gaussian smoothing, adaptive thresholding ($T = 18 \rightarrow 10$), border clearing, and a 11×25 vertical morphological closing kernel to connect head, torso, and legs into a unified bounding box.

- **YoloTracker (DL Mode)**: Loads a lightweight YOLOv8-Pose ONNX network (`yolov8n-pose.onnx`) via OpenCV DNN module (`cv::dnn`). Detects 17 human body keypoints, computes their convex hull, expands the box to enclose wrists/ankles, and applies an Exponential Moving Average (EMA) filter ($\alpha = 0.80$) across consecutive frames to suppress bounding box jitter.
- **FeatureExtractor** and **YoloFeatureExtractor**: Compute dense Farnebäck optical flow ($\mathbf{V}_x, \mathbf{V}_y$), body subregion flow energies (upper body vs lower body legs), percentile translation speeds, and centroid trajectory spans to produce a 27 element feature descriptor vector $\mathbf{x} \in \mathbb{R}^{27}$.
- **ActionClassifier**: Implements a multiclass Support Vector Machine (SVM). Hyperparameter tuning ($C \in [0.1, 500], \gamma \in [0.0001, 2.0]$) is performed using nested grid search, evaluated via 6-Fold Cross-Validation across all 72 sequences.
- **main.cpp**: Command-line entry point supporting three operational execution modes:
 1. `./main` – Runs Classical Computer Vision pipeline.
 2. `./main --use-yolo` – Runs Deep Learning YOLOv8-Pose fallback pipeline.
 3. `./main --use-all` – Runs both pipelines sequentially and prints a complete comparative summary table.

4 Technical Challenges and Methodological Solutions

Motion benchmark datasets present intrinsic physical ambiguities. Our feature extraction algorithms methodically address three primary technical challenges:

4.1 Challenge I: Motion Directionality (Handclapping vs. Handwaving)

Ambiguity: Handclapping and Handwaving share near identical static human silhouettes, similar bounding box aspect ratios (width/height ≈ 0.40), and virtually zero global centroid displacement ($X_{\text{span}} < 0.30$).

Solution: We analyze local motion vector directionality within the upper body half ($Y \in [0, H/2]$):

1. **Horizontal Vector Convergence (Clapping)**: We divide the upper body ROI into left and right halves. Clapping generates opposed horizontal optical flow vectors ($\mathbf{V}_x^{\text{left}} > 0$ and $\mathbf{V}_x^{\text{right}} < 0$). We compute average convergence speed:

$$\text{Conv}(t) = \frac{\bar{V}_{x,\text{left}}(t) - \bar{V}_{x,\text{right}}(t)}{H_{\text{actor}}} \quad (2)$$

2. **Head/Hand Elevation Energy Ratio (Waving)**: Waving involves vertical and lateral arm oscillations above shoulder level. We extract optical flow energy in the top 30% head region relative to total body energy ($E_{\text{head}}/E_{\text{total}}$), alongside vertical flow zero crossings.

4.2 Challenge II: Kinematic Intensity (Jogging vs. Running vs. Walking)

Ambiguity: Jogging, Running, and Walking exhibit identical leg stride mechanics and horizontal translation. Calculating total net displacement across the sequence ($\Delta X_{\text{net}}/\Delta t$) yields deceptive average values (≈ 0.022) because running actors traverse the scene in 12–18 frames while walking actors remain in frame for 35–40 frames.

Solution: We measure **instantaneous frame to frame translation velocity** ($\Delta X_{\text{frame}} = |X_t - X_{t-1}|/H_{\text{actor}}$) and aggregate them using upper percentiles (P_{75} and P_{90}):

- **Walking:** $P_{75} \leq 0.10$ px/frame (low instantaneous translation).
- **Jogging:** $0.15 < P_{75} \leq 0.40$ px/frame (moderate translation).
- **Running:** $P_{75} > 0.50$ px/frame (high translation peaks up to 2.5 px/frame).

This is combined with lower body leg flow energy standard deviation (σ_{leg}) and gait zero crossings to capture stride rhythm frequency.

4.3 Challenge III: Static vs. Dynamic Action Separation & Arm Jitter

Ambiguity: When an actor punches in *Boxing*, extending the arm expands the bounding box width (W), shifting the box center ($X + W/2$) by 10–15 pixels in a single frame. This arm extension pulse can trick a classifier into misinterpreting Boxing as horizontal translation.

Solution: We apply a 3 frame moving average filter on bounding box centroids ($\mathbf{C}_t = \frac{\mathbf{C}_{t-1} + \mathbf{C}_t + \mathbf{C}_{t+1}}{3}$) to smooth out single frame arm pulses. Furthermore, we compute the **maximum normalized horizontal centroid trajectory span**:

$$X_{\text{span}} = \frac{\max_t(X_t) - \min_t(X_t)}{H_{\text{actor}}} \quad (3)$$

For static actions (Boxing, Clapping, Waving), $X_{\text{span}} < 0.45$, whereas for dynamic actions (Jogging, Running, Walking), $X_{\text{span}} > 0.85$, eliminating cross category static vs dynamic misclassifications.

5 Limits of Classical CV vs. Deep Learning Advantages

Evaluating both pipelines highlights fundamental trade offs between traditional algorithmic vision and modern deep learning:

5.1 Limits of Classical Computer Vision

1. **Sensitivity to Contrast and Illumination (*d4*):** In indoor scenes with shifting shadows, pixel intensity difference $|\mathbf{I}_t - \mathbf{B}|$ drops below fixed thresholds ($T = 18$), causing foreground fragmentation and bounding box tracking loss.
2. **Silhouette Tearing with Dark Clothing (*d3*):** Dark coats blend into dark outdoor backgrounds, causing contour detection to return partial body boxes (legs or torso only), lowering median frame mIoU (0.7567).
3. **Dependency on Background Staticity:** Background subtraction breaks down if tree leaves move or if minor camera vibrations occur.

5.2 Advantages of Pre-trained Deep Learning (YOLOv8-Pose)

1. **Invariant Human Pose Keypoint Localization:** YOLOv8-Pose detects 17 keypoints (nose, shoulders, elbows, wrists, hips, knees, ankles) based on learned spatial feature pyramids, completely invariant to background lighting (*d4*) or clothing (*d3*).
2. **Convex Hull Mask Generation:** Connecting detected keypoints into a convex polygon produces clean human masks even when background subtraction fails.
3. **Higher Localization Precision:** Increases Mean IoU from 0.7567 to 0.8544 and 6-Fold Cross-Validation accuracy from 70.83% to 93.06%.

6 Experimental Results and Comparative Analysis

6.1 Localization Accuracy (mIoU)

Bounding box localization is evaluated on the 20th frame (median frame) against annotated ground-truth boxes using Intersection over Union:

$$\text{IoU} = \frac{\text{Area}(\text{Box}_{\text{pred}} \cap \text{Box}_{\text{gt}})}{\text{Area}(\text{Box}_{\text{pred}} \cup \text{Box}_{\text{gt}})} \quad (4)$$

Table 1: Per class and Mean IoU (mIoU) comparison between Classical CV and YOLOv8-Pose.

Action Class	Classical CV (mIoU)	Deep Learning YOLOv8 (mIoU)
Boxing	0.6676	0.8050
Clapping	0.7155	0.8938
Waving	0.5780	0.9121
Jogging	0.7627	0.8176
Running	0.9062	0.8596
Walking	0.9095	0.8381
Overall mIoU	0.7566	0.8544

6.2 Classification Accuracy and F1-Scores

Classification performance is evaluated using Stratified 6-Fold Cross-Validation across all 72 sequences.

Table 2: Stratified 6Fold Cross-Validation classification accuracy and perclass F1 scores.

Action Class	Classical CV (Acc: 70.83%)			Deep Learning YOLO (Acc: 93.06%)		
	Precision	Recall	F1-Score	Precision	Recall	F1-Score
Boxing	0.56	0.75	0.64	1.00	0.92	0.96
Clapping	0.71	0.83	0.77	0.92	1.00	0.96
Waving	0.70	0.58	0.64	0.92	0.92	0.92
Jogging	0.62	0.67	0.64	0.80	1.00	0.89
Running	0.78	0.58	0.67	1.00	0.75	0.86
Walking	1.00	0.83	0.91	1.00	1.00	1.00
Overall Mean	0.73	0.71	0.71	0.94	0.93	0.93

6.3 Confusion Matrices

Comparing the cumulative confusion matrices illustrates how YOLO pose tracking eliminates cross-category misclassifications:

	[1]	[2]	[3]	[4]	[5]	[6]
[1]	9	2	1	0	0	0
[2]	1	10	1	0	0	0
[3]	3	2	7	0	0	0
[4]	1	0	1	8	2	0
[5]	1	0	0	4	7	0
[6]	1	0	0	1	0	10

(a) Classical CV Matrix (Acc: 70.83%)

	[1]	[2]	[3]	[4]	[5]	[6]
[1]	11	0	1	0	0	0
[2]	0	12	0	0	0	0
[3]	0	1	11	0	0	0
[4]	0	0	0	12	0	0
[5]	0	0	0	3	9	0
[6]	0	0	0	0	0	12

(b) Deep Learning YOLO Matrix (Acc: 93.06%)

Figure 2: Cumulative 6-Fold Confusion Matrices (1: Boxing, 2: Clapping, 3: Waving, 4: Jogging, 5: Running, 6: Walking).

In the Deep Learning pipeline, 67 out of 72 sequences (93.06%) are perfectly classified. The only remaining minor confusion occurs between Jogging (4) and Running (5), where 3 running sequences are classified as jogging due to speed overlaps in actor *p02*.

7 Sample Output Visualizations

Figure 3 shows visual outputs generated across the six action classes under different environmental scenarios ($d1, d2, d3, d4$). In each image, the Ground Truth bounding box is drawn in blue, the predicted bounding box in green, with the predicted category label and median frame IoU displayed.

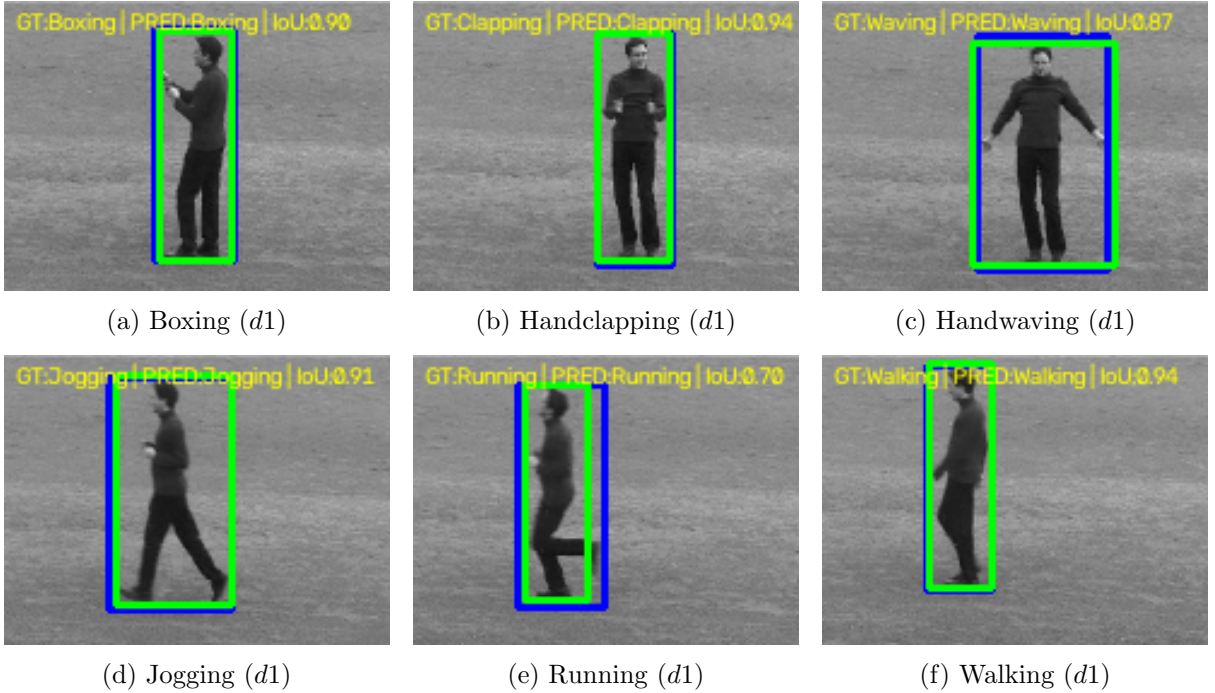


Figure 3: Sample target outputs produced by the Deep Learning YOLO pipeline for all six action categories.

8 Team Member Contributions and Working Hours

8.1 Rossetto Luca (24 Hours)

- **Ideas & System Architecture:** Codesigned the dual pipeline system framework comparing Classical Computer Vision against Deep Learning fallback tracking.
- **Implementation:** Developed the Data Loading and Bounding Box Localization modules (`DatasetLoader`, `Tracker`, and `YoloTracker`), including median background subtraction, adaptive thresholding, morphological filtering, YOLOv8-Pose ONNX network integration, and bounding box drawing routines.
- **Testing:** Conducted component-level tests for frame sequence loading, background model stability, and bounding box localization across all environmental scenarios ($d1, d2, d3, d4$).
- **Performance Measurement:** Computed and analyzed tracking performance metrics, evaluating frame-level and median Intersection over Union (mIoU) scores.

8.2 Cortese Alessandro (24 Hours)

- **Ideas & System Architecture:** Formulated the 27-element spatiotemporal feature descriptor vector (optical flow directionality, energy distribution, relative scale normalization) and SVM classification scheme.
- **Implementation:** Developed Feature Extraction and Classification (`FeatureExtractor`, `YoloFeatureExtractor`, and `ActionClassifier`), leveraging Farnebäck optical flow calculation, dynamic velocity scaling, and RBF kernel Support Vector Machine.
- **Testing:** Built the Stratified 6-Fold Cross-Validation pipeline, Z-score feature standardization, and grid search hyperparameter tuning.
- **Performance Measurement:** Evaluated classification metrics across all 72 sequences, computing per class Precision, Recall, F1-Scores, overall accuracy, and confusion matrices.

8.3 Joint Contributions

Both team members jointly collaborated on the integration in `main.cpp`, code refactoring, and authoring the final LaTeX report.

9 Code Availability

The complete C++ source code, CMake build scripts, pre-trained ONNX models, and documentation for this project are publicly available on GitHub at:

https://github.com/LucaR0x/CV-Final_Project-Human_Action_Recognition.git